Premise

One defense mechanism of RobustRAG against document-poisoning attacks is to answer from retrieved passages in isolation and keep only keywords that appear in sufficient aggregation. While the authors offer certifiable guarantees of correctness via exhaustive search for about 30% of queries, we are interested in where uncertified queries can go wrong, specifically how cheaply the keyword voting mechanism can be subverted by an attacker that fabricates no new factual claims (unlike standard poisoning attacks such as PoisonedRAG (Zou et al., 2025), which inject fabricated passages to assert a target answer).

Under RobustRAG's threat model, an attacker injects up to some budget of documents that displace the lowest ranked benign ones. We explore two strategies for such an attacker to counter RobustRAG's voting mechanism.

- Promotion: Repeat a wrong answer that already appears in a noisy honest passage, as a distractor. The goal is to lift its count over the threshold so it competes with the correct answer.
- Suppression: Injected passages that merely answer with any content, raising the raw number of nonabstaining documents. The goal is to increase the voting threshold until a correct keyword with weak majority falls below it and is excluded.

These two methods are coupled, since every promotion also raises the number of nonabstaining documents and thus suppresses as well. Therefore, our comparison hones in on two distinct costs: attack budget and search complexity. Suppression is cheap to construct but expensive in attack budget, since each passage lifts the threshold only slightly, so it needs many passages to exclude a well-supported keyword. Promotion is cheap in attack budget but conditional, since a usable distractor may simply not exist for a fraction of queries. We therefore investigate the tradeoff between the feasibility and the effectiveness of the two methods.

Division of work:

Manya: Promotion track - analysis, labeling code, live attacks
Connor: Suppression track - analysis, labeling code, live attacks

Procedure + Timeline:

Week 1:

Cache the isolated answers: Set up RobustRAG with Mistral-7B-Instruct on an A100 and run isolated per-passage answering on each dataset, saving every passage's answer and extracted keywords. The, confirm our setup by reproducing the paper's clean and certified accuracy on RealtimeQA.

Week 1/2:

Profile each query: From the cached answers, record 4 quantities per query:

1) No. of answering passages (n_b)

2) Vote count of the correct keyword (c*)
3) Vote count of the strongest incorrect keyword (d)
4) Abstention count

Week 2/3:

Predict vulnerability: Using the threshold rule $\mu = \min(\alpha \cdot n, \beta)$, we compute for each query whether one injected passage suffices to (1) promote an incorrect keyword over the threshold, or (2) suppress a correct keyword by inflating the threshold. Each query is labeled promotion-vulnerable, suppression-vulnerable, both, or safe, across $k' \in \{1, 2, 3\}$ and the paper's α settings.

Week 3/4:

Validate with live attacks: Select ~40 questions that cover each type of case: some where we predicted a promotion attack would work and some a suppression attack, and some short-answer (RealtimeQA, NQ) and some long-form (Bio). For each, write a real fake passage, add it to that question's retrieved passages, and run the full RobustRAG pipeline. Then check that the attack we predicted actually changes the final answer, and whether it needed as few fake passages as predicted.